

Automated Parameter Extraction from Scientific Literature: An Exhaustive Analysis of OCR and Vision-Language Parsing Architectures

Introduction to the Scientific Document Parsing Paradigm

Extracting structured parameters (e.g., thermodynamic data, solid-state battery properties) from scientific PDFs is a major challenge because the format strips semantic information. The industry is shifting from classical Optical Character Recognition (OCR) to Multimodal Large Language Models (MLLMs) and Vision-Language Models (VLMs) that treat parsing as spatial reasoning. However, these modern models can hallucinate numerical data when visual cues are ambiguous.

Systemic Methodologies for Extrapolating Parameters

Extracting exact parameters requires specialized agentic frameworks that orchestrate document parsers with domain-specific logic:

- **PERELMAN:** An agentic framework for materials science that uses the Docling parser and domain-knowledge VLMs to extract battery metrics, achieving up to 95.83% accuracy for voltage parameters.
- **SLM-MATRIX:** Uses multi-agent collaborative reasoning among Small Language Models to cross-verify extracted physics and chemistry data, reaching 92.85% accuracy on bulk modulus datasets.
- **AutoPK:** Reconstructs and normalizes complex multi-header tables in pharmacology to extract pharmacokinetic data with minimal hallucination.
- **KM-GPT:** Processes clinical plot images and risk tables to digitize survival curves with 100% accuracy, solving the failures of text-only tools.

The Architectural Evolution of Document Parsing Systems

OCR architectures are divided into three main paradigms, each with distinct strengths:

- **Legacy OCR (e.g., Tesseract):** Extremely fast for simple text but fails completely on 2D spatial relationships like mathematical equations or subscripts.
- **Modular Pipelines (e.g., MinerU, Docling, Marker):** These systems separate layout detection from recognition. MinerU is the accuracy leader for complex academic layouts (dense tables/math) but requires heavy GPU processing. Docling is highly efficient for digital-born PDFs, outputting structured JSON for databases.
- **End-to-End VLMs (e.g., Nougat, olmOCR):** Map document pixels directly to markup

languages. olmOCR achieves high fidelity on complex layouts via unit-test-driven reinforcement learning, while earlier models like Nougat excel at formulas but are prone to catastrophic repeating loops.

Deconstructing the Multimodal Bottleneck: Tables and Mathematical Expressions

Standard text metrics are insufficient for evaluating tables and formulas:

- **Table Extraction:** Evaluated via Tree Edit Distance based Similarity (TEDS). Pipeline systems (MinerU) currently outperform general VLMs on complex table structures because VLMs tend to hallucinate grid boundaries.
- **Formula Recognition:** Evaluated using the Character Detection Metric (CDM), which uses 2D spatial matching instead of text syntax. Specialized tools (Mathpix, UniMERNet) vastly outperform general MLLMs like GPT-4o on rendering exact mathematical parameters.

The Benchmarking Landscape for Document Intelligence

Evaluating OCR for parameter extraction relies on rigorous structural benchmarks:

- **olmOCR-Bench:** Uses binary visual unit tests (e.g., table alignment, math rendering) to test structural fidelity.
- **OmniDocBench:** Tests across diverse formats, proving pipelines win on academic text while VLMs handle visual noise better.
- **PubMed-OCR & DocLayNet:** Specialized datasets for evaluating clinical trial layouts and bounding box precision.

Pathology of Parsing Models: Hallucinations and the Linguistic Crutch

Autoregressive decoders in VLMs (e.g., DeepSeek-OCR) suffer from a critical vulnerability known as the "linguistic crutch." Under high optical compression or on blurry scans, these models abandon visual evidence and use linguistic priors to statistically guess missing text. While this produces fluent text, it results in plausible but entirely hallucinated numeric parameters, poisoning scientific extraction pipelines. For safety-critical data, deterministic pipeline architectures are required.

Operational Trade-Offs: Cost, Throughput, and Infrastructure

- **Commercial APIs:** Services like AWS Textract or Mathpix charge per page, which can cost \$10 to \$50 per 1,000 pages when tabular capabilities are invoked.
- **Open-Source GPU Infrastructure:** Self-hosting tools like olmOCR can process 1 million pages for under \$200 in compute costs. However, latency is a factor: lightweight parsers process over 20 pages/second, while heavy pipelines like MinerU process less than 1 page/second.

Strategic Conclusions

End-to-end VLMs present hallucination risks for numeric data due to their reliance on linguistic guessing. Reliable parameter extraction from scientific PDFs requires deterministic, pipeline-based OCR (like MinerU) for structure, specialized recognizers for math, and agentic frameworks (like PERELMAN) for domain-specific cross-validation.

Implementation Guide: Direct Visual Extraction Pipeline on DGX Spark

To operationalize the extraction of pharmacokinetic parameters directly from an uploaded PDF without relying on a full Retrieval-Augmented Generation (RAG) database, practitioners can deploy a Direct Visual Extraction Pipeline. This architecture is heavily optimized for unified memory systems like the NVIDIA DGX Spark (128 GB LPDDR5x) and utilizes a two-stage "smart filter" approach.

1. System Architecture & Hardware Stack

- **Intra-Document Retriever (Smart Filter):** The pipeline utilizes ColQwen2.5-7B. This model requires approximately 16GB of VRAM in FP16 precision¹. It embeds document pages into highly efficient visual patches; at its default resolution, it generates approximately 196 patches per page, consuming only about 100KB of memory per page¹. On high-bandwidth GPU architectures like the B200, it achieves a throughput of roughly 48 pages per second, meaning a standard 30-page PDF can be embedded in under 3 seconds¹.
- **Extraction VLM:** The extraction layer employs Qwen2.5-VL-7B (or a similarly sized multimodal model) served via the SGLang inference engine. SGLang enforces structured JSON generation via RadixAttention. Generation latency for extracting answers from a single page image ranges between 500ms and 2 seconds¹.
- **Data Processing:** NVIDIA cuDF is utilized to format the final Pandas DataFrame directly on the GPU, avoiding CPU bottlenecks.

2. Execution Flow

1. **Ingestion & Rasterization:** A FastAPI backend receives the PDF path and converts the document into images (e.g., at 200 DPI).
2. **Page Embedding & Intra-Document Search:** ColQwen2.5-7B rapidly embeds the page images. The system then embeds a search query (which takes 20 to 50ms) and executes a MaxSim vector search across the document (taking an additional 5 to 50ms) to identify the specific page containing the target data¹.
3. **Targeted Extraction:** The pipeline discards all irrelevant pages and routes only the highest-scoring image to the SGLang inference engine.
4. **GPU DataFrame Structuring:** The structured JSON output is loaded into cuDF for zero-copy GPU data formatting and returned to the user via the API response.

3. VLM Prompting Strategy & Schema

Because SGLang forces token generation to conform to a Pydantic schema, the prompt must

explicitly act as a data-entry instruction rather than a conversational prompt.

Pydantic Schema:

```
Python
from pydantic import BaseModel
from typing import List, Optional

class PKParameter(BaseModel):
    compound_name: str
    parameter_type: str
    value: float
    unit: str
    subject_species: Optional[str]

class ExtractedTable(BaseModel):
    parameters: List[PKParameter]
```

System Prompt:

You are an expert pharmacological data extraction system. You are provided with an image of a single page from a scientific paper. Your task is to identify tables or text containing pharmacokinetic parameters and extract them strictly into the requested JSON schema. Scan for Clearance, Half-life, Maximum Concentration, and Mean Residence Time. Extract the exact numerical value, unit, and compound. Do not hallucinate data. Return ONLY valid JSON matching the provided schema.

4. Code Implementation Outline (FastAPI + SGLang)

```
Python
from fastapi import FastAPI
from pydantic import BaseModel
import os

# Custom wrappers for ColQwen and SGLang assumed

app = FastAPI()

class ExtractionRequest(BaseModel):
    pdf_path: str
```

```

@app.post("/extract_pk/")
async def extract_pk_data(request: ExtractionRequest):
    # 1. Convert PDF to images
    images = convert_pdf_to_images(request.pdf_path)

    # 2. Run ColQwen2.5 filter to find the exact page
    page_embeddings = colqwen_model.embed_images(images)
    query_vector = colqwen_model.embed_query("Pharmacokinetic parameters table")
    best_page_index = calculate_maxsim(query_vector, page_embeddings)
    target_image = images[best_page_index]

    # 3. Extract data using SGLang with schema enforcement
    json_output = sglang_vlm.generate(
        image=target_image,
        prompt=SYSTEM_PROMPT,
        schema=ExtractedTable
    )

    # 4. Structure data with NVIDIA cuDF
    gdf = cudf.DataFrame(json_output['parameters'])
    gdf['parameter_type'] = gdf['parameter_type'].str.upper()

    return gdf.to_dict(orient="records")

```

Works cited

1. ColPali and Multimodal Document RAG on GPU Cloud - Spheron,
<https://www.spheron.network/blog/colpali-multimodal-document-rag-gpu-cloud/>